

Outil de pré-audit RGENS — mc2i

Dossier technique : architecture, choix, attendus IA & IHM

Juillet 2026 — document de synthèse

1. Vue d'ensemble

L'outil permet à un utilisateur de soumettre l'URL publique d'un site, d'obtenir un pré-audit automatisé selon les 42 critères du référentiel RGENS (Référentiel Général d'Écoconception de Services Numériques), de compléter les critères qui nécessitent une évaluation humaine, puis de comparer deux audits entre eux. L'application est volontairement scindée en deux services indépendants :

- **l'API** `mc2i-feature-api-go-rgesn` : calcule les audits (mesures automatiques, scoring, stockage) ;
- **le frontend** `rgesn-audit-frontend` : porte l'interface utilisateur et n'a aucune logique de scoring propre.

Cette séparation reflète l'organisation de l'équipe (un membre sur l'API/scoring, un sur le frontend + sécurité, un sur les fonctionnalités d'API) et permet à chaque service d'évoluer et d'être déployé indépendamment.

2. Architecture globale

Les deux services communiquent exclusivement en HTTP/JSON, serveur à serveur — le navigateur de l'utilisateur ne parle jamais directement à l'API :

Composant	Rôle	Port
Navigateur	Interface utilisateur (HTML + CSS + JS minimal)	—
Frontend (MVC)	Contrôleurs, gabarits HTML, appels à l'API	8081
API RGENS	Scoring, appels EcoIndex/Claude, scraping, stockage	8080
Services tiers	EcoIndex.fr (mesure environnementale), Claude (Anthropic)	externe

Schéma du flux : Navigateur → Frontend (MVC, port 8081) → API RGENS (port 8080) → EcoIndex.fr / Claude / navigateur headless Chrome (chromedp) → fichiers JSON (data/audits/) → réponse renvoyée à travers la même chaîne jusqu'au navigateur.

3. Détail des composants

3.1 API RGENS (`mc2i-feature-api-go-rgesn`)

Écrite en Go, sans framework web (net/http natif). Organisation par responsabilité :

- `internal/handlers` : points d'entrée HTTP (audit, suppression, export, critères, expert).
- `internal/rgesn` : le référentiel des 42 critères (`referentiel.go`), les évaluateurs automatiques déterministes (`mapping.go`), l'évaluation assistée par IA (`ia.go`) et le calcul des scores par thème (`scoring.go`).
- `internal/ecoindex` : client pour le service tiers EcoIndex.fr (poids de page, requêtes, score environnemental).
- `internal/scrapper` : pilote un Chrome headless (chromedp) pour mesurer ce qu'EcoIndex ne couvre pas (cache HTTP, formats d'image, lazy-loading, vidéos, extrait de scripts).

- `internal/claudeapi` : appelle l'API Claude pour les critères nécessitant une interprétation (UX/UI, contenus) — en best-effort, l'audit continue sans ces critères si la clé n'est pas configurée.
- `internal/security` : garde-fou anti-SSRF (`urlguard.go`), appelé avant tout appel réseau vers l'URL soumise, y compris pour chaque redirection.
- `internal/store` : persistance simple, un fichier JSON par audit, permissions restreintes au compte serveur.

Middlewares appliqués globalement : en-têtes de sécurité HTTP, CORS restreint, limitation de débit (10 requêtes/minute/IP) sur `/audit`, contrôle par jeton pour les endpoints d'export global.

3.2 Frontend (`rgesn-audit-frontend`)

Architecture MVC classique en Go, quatre pages seulement (conformément au cadrage du projet) :

Route	Contrôleur	Rôle
GET / , POST /audit	AccueilShow / AccueilCreateAudit	Saisie de l'URL, consentement RGPD, lancement de l'audit
GET /resume/{id}	ResumeShow	Scoring détaillé par thème/critère, complétude, complément expert
GET/POST /comparaison	ComparaisonShow / ComparaisonCompare	Comparaison de deux audits côte à côte
GET /info	InfoShow	Méthodologie, détail des 42 critères, politique de confidentialité

- `internal/apiclient` : seul point de contact avec l'API, avec cache mémoire de 10 minutes sur le référentiel des critères.
- `internal/viewmodel` : regroupement par thème, calcul de complétude/fiabilité.
- `internal/consentements` : journal RGPD des validations de la charte (SQLite), IP anonymisée, purge automatisable.
- `static/app.js` : amélioration progressive uniquement — confirmation avant suppression, indicateur de chargement, suppression automatique des données au départ de la page. Le site reste 100 % fonctionnel sans JavaScript.

3.3 Stockage

Aucune base de données lourde : un fichier JSON par audit côté API, une base SQLite unique et locale côté frontend pour le seul journal des consentements. Choix cohérent avec la sobriété recherchée : pas de serveur de base de données à faire tourner, sécuriser et alimenter en continu.

4. Cycle d'utilisation et flux technique

Cycle normal d'un utilisateur sur le site (il peut le recommencer entièrement à tout moment) :

1. L'utilisateur saisit une URL et statue sur la politique de confidentialité.

- S'il accepte : il passe à l'analyse.
- S'il refuse : il ne peut pas utiliser le site (le consentement est une condition d'accès, pas une option cochée par défaut).

2. EcoIndex analyse la page, à l'aide du scraper (`chromedp`) pour les mesures complémentaires (cache, formats d'image, lazy-loading...).

3. L'IA (Claude) complète l'analyse sur les critères UX/UI et Contenus, à partir des signaux techniques extraits (pas d'analyse visuelle à ce stade, cf. section 6).

4. Le scoring est calculé automatiquement (moyenne pondérée par thème).

- L'utilisateur peut s'arrêter là : télécharger le résultat en CSV, ou comparer directement avec un autre site.
- Ou il peut peaufiner le résultat en complétant le formulaire expert (étape 5).

5. Il ouvre un thème puis un critère, et répond via une barre range (avec justification libre optionnelle).

6. Sa réponse est intégrée à l'audit, qui recalcule le scoring — il peut revenir à l'étape 4 autant de fois qu'il le souhaite.

Il peut comparer directement avec l'URL de l'étape précédente, ou passer directement par la page Comparaison et saisir deux URLs indépendamment de tout audit déjà réalisé.

5. Choix techniques et justifications

Synthèse des arbitrages faits par l'équipe pendant le développement, avec leurs compromis assumés.

Langage et exécution

- **Go** a été choisi car il consomme peu de mémoire à l'exécution (compilé, sans machine virtuelle ni interpréteur lourd). Rust aurait pu être un choix encore plus économe, mais l'équipe manquait de connaissances et de temps pour se former dessus sur la durée du projet.

Mesure environnementale et collecte des données

- **EcoIndex** est utilisé en mode local (instance mc2i) ou via Docker : c'est le seul outil d'analyse 100 % automatique disponible.
- **Scraping vs API/Webhooks.** Le scraping (navigateur headless) a été retenu par manque de temps, mais reste un bon compromis. Avec plus de temps, une intégration par API/Webhooks aurait été une meilleure option d'un point de vue écologique (pas besoin de faire rendre une page complète pour en extraire quelques signaux).
- **Module IA interne plutôt que sous-traitant.** Construire un module d'IA propre évite de dépendre d'un sous-traitant supplémentaire qui recevrait les données. Cela suppose de bien connaître l'hébergement des modèles utilisés, avec comme mesure éco-responsable prioritaire la suppression des éléments transmis à l'IA une fois utilisés, et leur anonymisation avant envoi (cf. section 6).
- **Réduction des requêtes.** Le nombre de requêtes par paquet est diminué pour réduire l'impact environnemental du site, et le scoring est mis en cache le temps d'une session pour n'avoir à recalculer que l'ajout, pas l'intégralité de l'audit.

Hébergement

- **Render** est très simple à prendre en main (facile à maintenir), mais ne publie pas de rapport RSE et n'est donc pas considéré comme un hébergement vert.
- **AWS** est l'option la plus proche d'une démarche éco-responsable si l'on choisit une seule région de déploiement.
- **OVHcloud** est un bon compromis alternatif (hébergeur plus vert que Render), mais propose moins de services serverless.

Restitution et analyse (BI)

- **Power BI** est la meilleure option quand l'hébergeur cible n'est pas connu à l'avance.
- **Metabase / Superset** sont préférables si l'hébergeur est déjà vert (ex. OVHcloud), mais rendent la solution très dépendante de cet hébergeur.
- **Power Query** est un compromis simple à prendre en main (facile à maintenir) ; une solution plus écologique existe mais est nettement plus difficile à maintenir dans la durée.

Structure applicative

- **MVC en Go** a été choisi pour la sécurité et la facilité de maintenance. Ce n'est pas l'approche la plus standard pour un frontend, mais elle reste cohérente avec le langage utilisé côté API : un·e développeur·se qui connaît le Go peut intervenir aussi bien sur le frontend que sur le backend, avec seulement des notions de HTML/CSS/JS et une utilisation volontairement minimale du CSS/JS.
- **Bootstrap (en SaSS)** aurait pu être une bonne option pour la suite du développement, en évitant de charger des fichiers inutiles — ce qui facilite aussi une utilisation à l'échelle de l'UE et garde le code rapide.

Accessibilité

- **Difficulté par défaut fixée à "moyen"** pour rester proche d'une position neutre. Une piste d'amélioration identifiée : un formulaire détaillé qui adapte les questions suivantes en fonction des réponses déjà données.

Compte utilisateur

- **Pas de login pour l'instant.** Un compte permettant de conserver un audit (pour ne pas repartir de zéro) serait utile, mais l'arbitrage dépend de l'usage réel du site : conserver les audits en mémoire consomme en continu, alors que refaire plusieurs fois la même requête consomme davantage sur le moment — un compromis à trancher avec des données d'usage réelles.

No-code / low-code

- **Pas de no-code** : l'équipe a choisi de ne pas construire le site sur une plateforme no-code, pour garder la main sur la sécurité, s'assurer du respect de l'utilisateur, et limiter les risques liés au piratage. Le no-code est aussi plus difficile à maintenir sur le long terme (tout changement structurant oblige à recommencer à zéro).
- **Low-code assumé** : pour livrer un prototype dans les temps, l'équipe s'est appuyée au maximum sur des briques déjà prêtes (EcoIndex.fr, l'API Claude, chromedp) plutôt que de développer ces briques elle-même, afin de concentrer l'effort de développement sur la logique métier RGEN et sur le paramétrage de l'IA.

RGPD et déploiement

- **Consentement obligatoire** : une base dédiée à l'acceptation de la politique de confidentialité a été créée ; il est impossible d'utiliser le site sans cette acceptation.
- **Déploiement** : soit sur Docker (machine virtuelle sur site), soit en SaaS via Render (moins de contrôle, mais pas d'automatisation à gérer soi-même).

Portée de l'analyse

- **URL publique uniquement** : seules les URLs publiques sont acceptées, les accès privés étant rejetés — c'est l'usage général visé. Des pistes d'amélioration existent (lecture de base de données, de code source) mais demanderaient une sécurité et une analyse de risque supplémentaires avant d'être ouvertes.

Chatbot vs automatisation du scoring

- L'équipe n'a pas jugé intéressant de construire un chatbot ; l'effort a plutôt porté sur l'automatisation et le raffinement du scoring, pour produire davantage de résultats exploitables directement (cf. section 6 pour une réévaluation de ce choix au regard de l'attendu "assistant conversationnel").

6. Réponse à l'attendu IA & IHM low-code/no-code

Attendu : "Imaginer et prototyper avec une Interface Homme-Machine (IHM) claire et fonctionnelle, avec une solution low-code/no-code permettant de se concentrer sur le paramétrage et la logique IA." Ci-dessous, la réponse globale puis, pour chaque sous-attendu, un état honnête de ce qui est implémenté, partiel, ou à construire sous forme de module dédié.

6.1 IHM et posture low-code

L'IHM tient en quatre pages (Accueil, Résumé, Comparaison, Info), en HTML servi côté serveur, sans framework JavaScript : formulaires natifs, accordéons <details>/<summary>, barres de progression en CSS pur, et un fonctionnement garanti même JavaScript désactivé. C'est une IHM volontairement sobre, cohérente avec ce que l'outil évalue lui-même chez les sites audités.

Sur le "low-code/no-code" : l'équipe a choisi de ne pas passer par une plateforme no-code (raisons de sécurité et de maintenabilité, cf. section 5), mais applique l'esprit de l'attendu autrement — en s'appuyant sur des services et bibliothèques existants (EcoIndex.fr, Claude, chromedp) plutôt qu'en réinventant ces briques, l'essentiel du code écrit à la main porte sur la logique RGEN (référentiel, pondération, scoring) et sur le paramétrage de l'IA (prompts, choix des critères confiés au modèle), pas sur de la plomberie technique générique.

6.2 Sous-attendus, un par un

[Implémenté] Anonymisation avant envoi au modèle IA

Le module IA (`internal/rge/ia.go`) ne transmet à Claude que des signaux techniques agrégés : liste des vidéos, extrait des scripts inline, nombre total d'images. Ni l'URL auditée, ni les réponses en texte libre de l'utilisateur, ni aucune donnée personnelle ne sont envoyées — cohérent avec la politique de confidentialité (section 5, destinataires).

[Partiel] Traitement local

EcoIndex peut tourner en mode local ou via Docker, et le stockage des audits est entièrement local (fichiers JSON, aucune base cloud). En revanche, l'interprétation par l'IA reste par nature externalisée : Claude est un modèle hébergé chez Anthropic, pas auto-hébergé. Un modèle local (petit LLM open-source auto-hébergé) est une piste identifiée mais non retenue, faute de temps et de puissance de calcul disponible pendant le sprint.

[Implémenté] Politique de minimisation

Seules l'URL publique, les mesures techniques automatiques et les réponses libres optionnelles sont collectées ; aucun identifiant ni mot de passe. Détaillé dans la politique de confidentialité (section 3) et appliqué de bout en bout : rien n'est demandé "au cas où".

[Partiel] Journalisation et traçabilité

Le consentement à la charte est journalisé (date, heure, IP anonymisée) dans une base séparée, à titre de preuve. Les erreurs applicatives sont journalisées côté serveur. Il n'existe en revanche pas de journal d'audit détaillé "qui a consulté/modifié quel audit" — cohérent avec l'absence de comptes utilisateurs, mais à revoir si un login est ajouté par la suite (cf. section 5).

[Implémenté] Consentement éclairé

Case à cocher obligatoire et non pré-cochée avant tout lancement d'audit, avec lien vers la politique complète ; impossible d'utiliser le site sans cette validation ; consentement retirable à tout moment (droits RGPD, section 7).

[Implémenté] Conformité RGPD

Traité de façon transversale : base légale (consentement), minimisation, durée de conservation resserrée (1 an pour le journal de consentement, suppression automatique des résultats d'audit à la sortie de la page), droits de la personne, mesures de sécurité (SSRF, en-têtes HTTP, permissions de fichiers, accès aux exports par jeton), transparence sur les transferts hors UE vers Anthropic.

[Module à créer] Injection facile de contexte

Le prompt envoyé à l'IA (`evaluerThemeIA`) est aujourd'hui fixe et identique pour tous les sites : mêmes signaux techniques, même trame de questions. Il n'existe pas de champ permettant d'injecter un contexte métier (secteur d'activité, type de site — vitrine, e-commerce, SaaS...) qui affinerait l'interprétation. C'est un module à ajouter (cf. 6.3).

[Implémenté] Matrice de notation dynamique basée sur RGEN

La méthode `Critere.Coefficient()` croise Priorité × Difficulté (ex. Prioritaire+Faible = 5, Recommandé+Fort = 2) pour pondérer chaque critère, exactement selon les règles du document GPS V2 du RGEN. Le score par thème (`CalculerScoresThemes`) est recalculé automatiquement à chaque nouvelle réponse (formulaire expert ou IA) : la matrice est donc bien dynamique, pas figée à la génération de l'audit.

[Module à créer] Ingestion et analyse de fichiers par l'IA

Identifiée dès la conception (l'équipe avait prévu qu'un fichier PDF envoyé serait compressé avant analyse) mais non construite faute de temps : il n'existe aujourd'hui aucun point d'entrée pour uploader un fichier. C'est un module à ajouter (cf. 6.3).

[Module à créer] Assistant conversationnel (questions de contexte et de qualification)

Choix assumé de ne pas construire de chatbot : un modèle conversationnel consomme sensiblement plus qu'un formulaire structuré pour un bénéfice incertain, ce qui allait à l'encontre de la sobriété recherchée. L'équipe a préféré concentrer l'effort sur l'automatisation du scoring. Ce choix reste défendable, mais ne répond pas à l'attendu tel quel — une version allégée (questionnaire de qualification structuré plutôt qu'un chat libre) est proposée en 6.3 comme compromis plus sobre.

[Implémenté] Module d'analyse comparative entre différents sites web ou applications

La page Comparaison (`ComparaisonShow` / `ComparaisonCompare`) affiche deux audits côte à côte, thème par thème et critère par critère, accessible depuis le Résumé ("Comparer avec un autre site") ou directement en saisissant deux URLs.

[Partiel] Génération automatique de rapport de sortie

Un export CSV détaillé par critère existe ("Télécharger en CSV" sur le Résumé). Il n'y a en revanche pas encore de génération automatique d'un rapport de synthèse formaté (PDF/Word) directement depuis l'outil — ce document en est un exemple, mais produit manuellement. La piste d'automatisation est décrite en 6.3.

6.3 Pour aller plus loin : le module d'analyse IA avancé

L'IA ne couvre aujourd'hui que les critères UX/UI et Contenus, à partir de signaux techniques limités (vidéos, extrait de script, nombre d'images) : pas d'analyse visuelle réelle, pas de contexte métier, pas de fichiers, pas d'échange avec l'utilisateur. Pour répondre pleinement aux sous-attendus encore ouverts, un module dédié (proposé : un package séparé, par exemple `internal/iavancee` côté API) serait nécessaire, avec :

- **Injection de contexte** : un champ "secteur d'activité / type de site" renseigné par l'utilisateur (ou déduit d'un court questionnaire de qualification), anonymisé puis transmis dans le prompt pour affiner l'interprétation IA.
- **Ingestion de fichiers** : upload sécurisé d'un document (cahier des charges, export de code, PDF), avec compression et extraction de texte anonymisée avant tout envoi au modèle, et une limite de taille stricte pour rester sobre.
- **Analyse visuelle** : capture d'écran anonymisée (floutage des zones de texte personnel) analysée par un modèle multimodal, pour juger réellement l'ergonomie (critères UX/UI) plutôt que par des proxys techniques indirects.
- **Qualification structurée plutôt qu'un chat libre** : quelques questions fermées posées avant l'audit (pas un assistant conversationnel complet), qui orientent l'IA — un compromis plus sobre, cohérent avec l'esprit RGEN, tout en répondant à l'intention de l'attendu (qualifier le contexte auprès de l'utilisateur).
- **Rapport automatique** : réutilisation du même moteur de génération de document que ce dossier (score, synthèse par thème, actions recommandées déjà calculées par l'outil), déclenché en un clic depuis le Résumé plutôt que produit à la main.

Ce module n'a pas été construit dans le temps imparti pour trois raisons : le temps de développement disponible, le coût et la puissance de calcul supplémentaires (plus de tokens envoyés par audit, potentiellement un modèle multimodal plus lourd), et une vigilance RGPD à renforcer en proportion — chaque nouvelle donnée transmise à l'IA (fichier, capture d'écran, réponses de qualification) doit repasser par la même grille de minimisation et d'anonymisation que les signaux actuels, pas être ajoutée par défaut.

7. Pourquoi ce choix d'architecture sert le RGEN

- **Rendu serveur, pas de framework JS lourd.** HTML généré côté serveur (`html/template`), sans bundle JS à télécharger ; le seul script fait quelques kilo-octets et est chargé sans bloquer l'affichage.
- **Fonctionnement sans JavaScript.** Formulaire natifs, validation HTML5, accordéons `<details>/<summary>` : la dégradation gracieuse est un principe directeur du RGEN, pas une option secondaire.

- **Go compilé plutôt qu'un framework interprété.** Empreinte mémoire et CPU réduite par requête, dépendances minimales.
- **Pas de sondage réseau superflu.** L'indicateur d'attente pendant l'analyse est un chronomètre local, pas de polling AJAX répété.
- **Mise en cache.** Référentiel des critères mis en cache 10 minutes côté frontend, fichiers statiques servis avec Cache-Control.
- **Synthèse frugale.** La vue de synthèse par thème sur le Résumé est en CSS pur, sans librairie de graphique JS.
- **Stockage minimal.** Fichiers JSON plutôt qu'un SGBD permanent ; suppression automatique des résultats à la sortie de la page.

L'outil pratique ainsi une forme de cohérence : il serait délicat de recommander l'écoconception aux sites audités avec un outil qui ne l'appliquerait pas à lui-même.

8. Pourquoi ce choix d'architecture sert le RGPD

- **Minimisation des données.** Seule une URL publique est traitée ; aucun identifiant ni mot de passe n'est jamais collecté.
- **Suppression automatique.** Les résultats d'un audit sont supprimés dès la sortie de la page Résumé (`navigator.sendBeacon` sur `pagehide`), sauf pendant une comparaison active.
- **Anonymisation du journal de consentement.** IP tronquée avant stockage, conservée 1 an, avec purge automatisable.
- **Protection contre le SSRF.** Chaque URL, et chaque redirection suivie par le navigateur headless, est validée contre les plages IP privées et réservées avant tout appel réseau.
- **Surface d'attaque réduite.** Endpoints d'export global protégés par jeton et désactivés par défaut ; CORS restreint à une origine explicite.
- **Défense en profondeur générique.** En-têtes de sécurité HTTP, limitation de débit par IP, permissions de fichiers restreintes, messages d'erreur génériques côté client.
- **Transparence.** Politique de confidentialité accessible depuis chaque page, avec le détail des destinataires et des transferts hors UE.

9. Limites connues et pistes d'amélioration

- Le contrôle SSRF revalide chaque redirection observée par le navigateur headless, mais reste basé sur une résolution DNS séparée de celle du navigateur : une attaque par DNS rebinding fine reste théoriquement possible.
- Le stockage en fichiers JSON convient au développement et à un usage local, mais devrait migrer vers une base de données réelle avant un déploiement à plusieurs utilisateurs simultanés.
- HTTPS n'est pas géré nativement par les serveurs Go ; à ajouter via un reverse-proxy/certificat au moment d'un déploiement réel exposé publiquement.
- Le transfert de données vers Claude (Anthropic) hors Union européenne nécessite des clauses contractuelles types validées par un DPO avant toute utilisation en production.
- La purge du journal des consentements existe mais n'est pas encore branchée à une tâche planifiée automatique.
- Trois sous-attendus IA restent à construire sous forme de module dédié : injection de contexte, ingestion de fichiers, qualification structurée de l'utilisateur (cf. section 6.3).
- La génération de rapport de sortie n'est aujourd'hui qu'un export CSV, pas encore un document de synthèse formaté généré automatiquement.
- Pas de compte utilisateur : un audit non terminé se perd si la page est quittée sans avoir été conservé via la comparaison — arbitrage assumé entre confort et sobriété (cf. section 5).

Annexe — Planning de réalisation

Jour	Prévu	Réalisé	Annexe
Lundi	Commencement du code ; répartition des rôles ; plan de structure ; pré-code	Rôles presque tous déterminés ; début de code pour tout le monde ; architecture solide et modulaire ; pré-code terminé	Présentation du projet le matin
Mardi	Test basique du site ; recherche des failles et du parcours utilisateur	Scraping terminé et lié à EcoIndex ; frontend « terminé » en attente du backend ; failles et actions client identifiées	Cours sécurité 8h–10h
Mercredi	Amélioration du premier prototype ; sécurité ; ajout d'options (low-code)	Scraping-EcoIndex-Scoring-IA liés dans une API ; début d'argumentation du site et du pitch ; partie expert terminée et intégrée au frontend ; travaux fusionnés	
Jeudi	Préparation du pitch ; peaufinage du site et de l'argumentation	Peaufinage du site et dernière version ; préparation du support ; répartition de la prise de parole	
Vendredi	Ne plus toucher au site ; peaufinage de la présentation		Présentation des projets au jury